Assignment: Concepts, Syntax and Lexical Analysis
Completed by: Cailiang Hua, 000931700

Part B: Syntax Description

1. Problem Statement: Finding the Average
    a. Given a set of numbers, how can we find their average?
    1. Calculator the sum of the numbers.
    2. Calculator the length(N) of the numbers.
    3. Using sum divided by list's length(N) and then getting the average

    b. Example: Given the numbers [5, 8, 12, 4, 10], calculate the average.
    1. (5 + 8 + 12 + 4 + 10)/5 = 7.8

2. Develop pseudo code to solve the problem statement outlined in Step 1.
    Use descriptive variable names and comments to enhance readability and understanding.

    ```
    //setting default list
    list <- [5, 8, 12, 4, 10]
    //calculator list sum
    listSum <- 0
    For J <- 0 to list.length - 1 Do
        listSum <- list[J] + listSum
    End For
    //Calculator average
    listAverage <-  listSum/list.length
    ```

3. Write a Backus-Naur Form (BNF) grammar to describe the syntax of the problem statement.

    <program> ::= <statement-list>
    <statement-list> ::= <statement> | <statement> <statement-list>
    <statement> ::= <expr> | “for” <expr> “to” <expr> { <statement-list> }
    <expr> ::= <expr> | <expr> "=" <term> | <expr> “=” <expr> | <expr> “+” <term> | <expr> “-” <term> | <term> “/” <term>
    <term> ::= <factor>
    <factor> ::= <id> | <number> | <array-list> | <array-pop> | <array-length>
    <array-list> ::= [<number-list>]
    <number-list> ::= <number> | <number>, <number-list>
    <array-pop> ::= <id>[<expr>]
    <array-length> ::= <id>.” length”
    <number> ::= <digit> | <digit> <number>
    <digit> ::= “0” | “1” | “2” | “3” | “4” | “5” | “6” | “7” | “8” | “9”
    <id> ::= <letter> | <letter> <id>
    <letter> ::= “a” |” b” |” c” |” d” |” e” |” f” |” g” |” h” |” I” |” j” |” k” |” l” |” m” |” n” |” o” |” p” |” q” |” r” |” s” |” t” |” u” |” v” |” w” |” x” |” y” |” z”

|" A" |" B" |" C" |" D" |" E" |" F" |" G" |" H" |" I" |" J" |" K" |" L" |" M" |" N" |" O" |" P" |" Q" |" R" |" S" |" T" |" U" |" V" |" W" |" X" |" Y" |" Z"

4. Construct a parse tree based on the BNF grammar to visualize the syntactic structure of the problem.
    1. Seeing the image “CailiangHua3BNFParseTree.png”

5. Examine the parse tree constructed in Step 4 to determine if the BNF grammar is ambiguous.
    1. Identify any areas where multiple interpretations are possible.
        The section “<expr> ::= <expr> | <expr> "=" <term> | <expr> “=” <expr> | <expr> “+” <term> | <expr> “-” <term> | <term> “/” <term>” has multiple interpretations.
    2. Provide examples or scenarios to illustrate potential ambiguities and their implications.
        For examples “<expr> ::= <expr> | <expr> "=" <term> | <expr> “=” <expr>”, it can be <expr> "=" <term> or <expr> "=" <term>, it’s hard to identify which expression should we use.

6. Rewrite the BNF grammar using Extended Backus-Naur Form (EBNF) to enhance readability and expressiveness.
    1. Explain the benefits of using EBNF over traditional BNF, particularly in terms of readability and clarity.

        BNF relies on recursion to express repetition, whereas EBNF relies on braces.
        When writing combinations of expressions, BNF requires multiple definitions, whereas EBNF simply uses parentheses for grouping.
2. Remove any ambiguity present in the BNF grammar through syntax modifications in the EBNF version.

    program ::= statement-list;
    statement-list ::= statement { statement };
    statement ::= expr | “for” expr “to” expr { statement-list }; | equal-expr | addition-expr | subtraction-expr | multiplication-expr | division-expr | equal-addition-expr | equal-division-expr
    equal-expr ::= term “=” term
    addition-expr ::= term “+” term
    subtraction-expr ::= term “-” term
    multiplication-expr ::= term “/” term
    equal-addition-expr ::= term “=” term “+” term
    equal-division-expr ::= term “=” term “/” term
    term ::= factor;
    factor ::= id | number | array-list | array-pop | array-length;
    array-list ::= [number-list];
    number-list ::= number { “,” number };
    array-pop ::= id[expr];

array-length ::= id.” length”;
number ::= digit { digit };
digit ::= “0” | “1” | “2” | “3” | “4” | “5” | “6” | “7” | “8” | “9”
id ::= letter { letter };
letter ::= “a” |” b” |” c” |” d” |” e” |” f” |” g” |” h” |” I” |” j” |” k” |” l” |” m” |” n” |” o” |” p” |” q” |” r” |” s” |” t” |” u” |” v” |” w” |” x” |” y” |” z”
|” A” |” B” |” C” |” D” |” E” |” F” |” G” |” H” |” I” |” J” |” K” |” L” |” M” |” N” |” O” |” P” |” Q” |” R” |” S” |” T” |” U” |” V” |” W” |” X” |” Y” |” Z”

7. Use the EBNF grammar to reconstruct a parse tree, demonstrating how the revised syntax resolves any ambiguities present in the original BNF grammar.
    a. Analyze and compare the parse trees from Steps 4 and 7 to understand the impact of syntax modifications on the parse tree structure.
    Seeing the image “CailiangHuaEBNFParseTree.png”
    After comparing the parse trees from steps4 and 7, the EBNF seems more simpler than the BNF, especially on the recursion to express repetition.